

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 7**



TREE (POHON)

Oleh:
Aufa Thariq Akram 2510817310014

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 7

Laporan Praktikum Algoritma & Struktur Data Modul 7: Tree (Pohon) ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	3
B. Output Program	4
C. Pembahasan	10
LINK GITHUB	13

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Program	8
Gambar 2 Opsi 1 Tambah Data 2 4 6 8 10 11 13.....	8
Gambar 3 Opsi 2 PreOrder.....	8
Gambar 4 Opsi 3 inOrder	9
Gambar 5 Opsi 4 PostOrder	9
Gambar 6 Opsi 5 Exit.....	9

DAFTAR TABEL

Tabel 1 Source Code Soal 1	4
----------------------------------	---

SOAL 1

Cobalah program berikut, perbaiki output, lengkapi fungsi inOrder dan postOrder pada coding, running, simpan program !

```
1  #include <stdio.h>
2  #include <conio.h>
3  #include <stdlib.h>
4  #include <iostream>
5
6  using namespace std;
7  struct Node
8  {
9      int data;
10     Node *kiri;
11     Node *kanan;
12 };
13
14 void tambah(Node **root, int databaru)
15 {
16     if (*root == NULL)
17     {
18         Node *baru;
19         baru = new Node;
20         baru->data = databaru;
21         baru->kiri = NULL;
22         baru->kanan = NULL;
23         (*root) = baru;
24         (*root)->kiri = NULL;
25         (*root)->kanan = NULL;
26         cout << "Data bertambah";
27     }
28     else if (databaru < (*root)->data)
29         tambah(&(*root)->kiri, databaru);
30     else if (databaru > (*root)->data)
31         tambah(&(*root)->kanan, databaru);
32     else if (databaru == (*root)->data)
33         cout << "Data sudah ada";
34 }
35
36 void preOrder(Node *root)
37 {
38     if (root != NULL)
39     {
40         cout << root->data;
41         preOrder(root->kiri);
42         preOrder(root->kanan);
43     }
44 }
45
46 void inOrder(Node *root)
47 {
48     if (root != NULL)
```

```

49
50
51
52
53
54 }
55
56 void postOrder(Node *root)
57 {
58
59
60
61
62
63
64 }
65
66 int main()
67 {
68     int pil, data;
69     Node *pohon;
70     pohon = NULL;
71     do
72     {
73         system("cls");
74         cout << "1. Tambah\n";
75         cout << "2. PreOrder\n";
76         cout << "3. inOrder\n";
77         cout << "4. PostOrder\n";
78         cout << "5. Exit\n";
79         cout << "\nPilihan : ";
80         cin >> pil;
81         switch (pil)
82         {
83             case 1:
84                 cout << "\n INPUT : ";
85                 cout << "\n -----";
86                 cout << "\n Data baru : ";
87                 cin >> data;
88                 tambah(&pohon, data);
89                 break;
90             case 2:
91                 cout << "PreOrder";
92                 cout << "\n-----\n";
93                 if (pohon != NULL)
94                 {
95                     preOrder(pohon);
96

```

```

97         else
98             cout << "Masih Kosong";
99         break;
100     case 3:
101         cout << "InOrder";
102         cout << "\n-----\n";
103         if (pohon != NULL)
104         {
105             inOrder(pohon);
106         }
107         else
108             cout << "Masih Kosong";
109         break;
110     case 4:
111         cout << "PostOrder";
112         cout << "\n-----\n";
113         if (pohon != NULL)
114         {
115             postOrder(pohon);
116         }
117         else
118             cout << "Masih Kosong";
119         break;
120     case 5:
121         return 0;
122     }
123     _getch();
124 } while (pil != 5);
125 return EXIT_FAILURE;
126 }

```


A. Source Code

Tabel 1 Source Code Soal 1

1	#include <stdio.h>
2	#include <stdlib.h>
3	#include <iostream>
4	#include <termios.h>
5	#include <unistd.h>
6	
7	using namespace std;
8	
9	int getch() {
10	struct termios oldt, newt;
11	int ch;
12	tcgetattr(STDIN_FILENO, &oldt);
13	newt = oldt;
14	newt.c_lflag &= ~(ICANON ECHO);
15	tcsetattr(STDIN_FILENO, TCSANOW, &newt);
16	ch = getchar();
17	tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
18	return ch;
19	}
20	
21	struct Node {
22	int data;
23	Node *kiri;
24	Node *kanan;
25	};
26	
27	void tambah(Node **root, int databaru) {
28	if (*root == NULL) {
29	Node *baru = new Node;
30	baru->data = databaru;
31	baru->kiri = NULL;
32	baru->kanan = NULL;
33	(*root) = baru;
34	cout << "Data bertambah";
35	} else if (databaru < (*root)->data) {
36	tambah(&(*root)->kiri, databaru);
37	} else if (databaru > (*root)->data) {
38	tambah(&(*root)->kanan, databaru);
39	} else {
40	cout << "Data sudah ada";
41	}
42	}

```

43
44 void preOrder(Node *root) {
45     if (root != NULL) {
46         cout << root->data << " ";
47         preOrder(root->kiri);
48         preOrder(root->kanan);
49     }
50 }
51
52 void inOrder(Node *root) {
53     if (root != NULL) {
54         inOrder(root->kiri);
55         cout << root->data << " ";
56         inOrder(root->kanan);
57     }
58 }
59
60 void postOrder(Node *root) {
61     if (root != NULL) {
62         postOrder(root->kiri);
63         postOrder(root->kanan);
64         cout << root->data << " ";
65     }
66 }
67
68 int main()
69 {
70     int pil, data;
71     Node *pohon;
72     pohon = NULL;
73     do
74     {
75         system("clear");
76         cout << "1. Tambah\n";
77         cout << "2. PreOrder\n";
78         cout << "3. inOrder\n";
79         cout << "4. PostOrder\n";
80         cout << "5. Exit\n";
81         cout << "\nPilihan : ";
82         cin >> pil;
83         switch (pil)
84         {
85             case 1:
86                 cout << "\n INPUT : ";
87                 cout << "\n -----";

```

88	cout << "\n Data baru : ";
89	cin >> data;
90	tambah(&pohon, data);
91	cout << "\n";
92	break;
93	case 2:
94	cout << "\nPreOrder";
95	cout << "\n-----
	\n";
96	if (pohon != NULL)
97	{
98	preOrder(pohon);
99	cout << "\n";
100	}
101	else
102	cout << "Masih Kosong\n";
103	break;
104	case 3:
105	cout << "\ninOrder";
106	cout << "\n-----
	\n";
107	if (pohon != NULL)
108	{
109	inOrder(pohon);
110	cout << "\n";
111	}
112	else
113	cout << "Masih Kosong\n";
114	break;
115	case 4:
116	cout << "\nPostOrder";
117	cout << "\n-----
	\n";
118	if (pohon != NULL)
119	{
120	postOrder(pohon);
121	cout << "\n";
122	}
123	else
124	cout << "Masih Kosong\n";
125	break;
126	case 5:
127	return 0;
128	}
129	getch();

130	} while (pil != 5);
131	return EXIT_FAILURE;
132	}

B. Output Program

```
MacBook-Air-AUFA:Coding aufathariqakram$ cd "/Users/aufathariqakram/g/C:C++ progamming/c++ folder/Modul 7/"Tree
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : █
```

Gambar 1 Tampilan Awal Program

```
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 2
```

Gambar 2 Opsi 1 Tambah Data 2 4 6 8 10 11 13

```
Pilihan : 2

PreOrder
-----
2 4 6 8 10 11 13
```

Gambar 3 Opsi 2 PreOrder

```
Pilihan : 3  
  
inOrder  
-----  
2 4 6 8 10 11 13
```

Gambar 4 Opsi 3 inOrder

```
Pilihan : 4  
  
PostOrder  
-----  
13 11 10 8 6 4 2
```

Gambar 5 Opsi 4 PostOrder

```
1. Tambah  
2. PreOrder  
3. inOrder  
4. PostOrder  
5. Exit  
  
Pilihan : 5  
○ MacBook-Air-AUFA:Modul 7 aufathariqakram$ █
```

Gambar 6 Opsi 5 Exit

C. Pembahasan

baris 1 - 7, syntax `#include <stdio.h>`, `#include <stdlib.h>`, `#include` , `#include <termios.h>`, dan `#include <unistd.h>` berfungsi untuk mengintegrasikan cetak biru pustaka bawaan dari bahasa C/C++ serta antarmuka sistem operasi berbasis UNIX ke dalam program yang sedang dibangun. pustaka `iostream` secara fundamental mengaktifkan jalur komunikasi untuk urusan input-output standar di layar, sedangkan `stdio.h` dan `stdlib.h` turut dipanggil untuk mendukung operasi sistem dasar seperti membersihkan layar. sementara itu, `termios.h` dan `unistd.h` merupakan pustaka antarmuka tingkat rendah yang krusial untuk ekosistem macOS guna memanipulasi bagaimana ketikan keyboard diproses secara langsung oleh terminal. adapun syntax `using namespace std;` adalah instruksi direktif kepada kompiler agar program secara otomatis mencari semua fungsi standar di dalam ruang nama `std`, sehingga kode di bawahnya terbebas dari penulisan awalan `std::` yang berulang-ulang.

baris 9 - 20, syntax `int getch()` berfungsi untuk mendefinisikan sebuah fungsi penangkap karakter kustom tingkat rendah yang diciptakan sebagai solusi kompatibilitas absolut karena pustaka `conio.h` milik Windows tidak diakui oleh kernel macOS. fungsi ini beroperasi dengan teknik manipulasi atribut terminal menggunakan struktur data `termios`, di mana ia pertama-tama akan menyalin konfigurasi terminal asli ke dalam variabel `oldt` dan `newt` untuk dicadangkan. ia kemudian memodifikasi variabel `newt` dengan mematikan secara paksa bendera `ICANON` (mode per-baris) dan `ECHO` (penampil karakter) agar ketikan pengguna tidak menempel di layar, lalu menyuntikkan pengaturan tersebut melalui perintah `tcsetattr`. hal ini memungkinkan fungsi `getchar()` bereaksi seketika saat satu tombol ditekan tanpa menunggu instruksi enter, sebelum akhirnya program memulihkan konfigurasi terminal ke wujud aslinya agar sistem tidak mengalami malfungsi.

baris 22 - 26, syntax `struct Node` berfungsi untuk mendefinisikan fondasi atau struktur data kustom yang menjadi cetak biru utama pembentuk pohon biner. blok ini bertindak sebagai wadah memori node tunggal yang memuat tiga komponen esensial: sebuah variabel integer bernama `data` untuk menampung nilai angka aktual, serta dua buah variabel pointer bernama `kiri` dan `kanan` yang bertugas sebagai jangkar navigasi untuk

mengaitkan node tersebut dengan anakan di sebelah kiri maupun kanannya di dalam ruang memori.

baris 28 - 43, syntax void tambah(Node root, int databaru) berfungsi untuk mengkapsulasi blok logika rekursif yang bertugas menyisipkan elemen data baru ke dalam struktur pohon biner. fungsi ini secara spesifik menerima parameter double pointer atau pointer ganda root agar dapat secara langsung memodifikasi alamat memori asli dari pohon tersebut. mekanisme kerjanya dimulai dengan memeriksa apakah akar saat ini kosong (NULL); jika ya, program akan mengalokasikan memori baru dengan perintah new Node, mengisi nilainya, dan memutus rantai pointer anaknya menjadi NULL. namun jika node sudah berisi data, algoritma akan melakukan komparasi nilai: menelusuri cabang secara rekursif ke arah kiri jika data baru lebih kecil, atau ke arah kanan jika data baru lebih besar, dan akan memunculkan pesan penolakan "Data sudah ada" apabila terdeteksi adanya duplikasi nilai.

baris 45 - 67, syntax void preOrder(Node *root), void inOrder(Node *root), dan void postOrder(Node *root) berfungsi untuk menyelenggarakan tiga metode penelusuran (traversal) standar yang diwajibkan dalam teori pohon biner menggunakan ketajaman algoritma rekursif. fungsi preOrder bekerja dengan mencetak nilai node induk terlebih dahulu sebelum menyelam lebih dalam ke anak kiri dan anak kanannya. fungsi inOrder menelusuri secara ekstrem ke ujung anak kiri terlebih dahulu, mencetak nilai node saat perjalanan kembali ke atas, lalu beralih ke anak kanan, yang mana proses ini secara otomatis akan menghasilkan urutan angka yang tersortir dengan rapi. sedangkan fungsi postOrder beroperasi dengan cara menguras dan menelusuri seluruh anak kiri dan kanan hingga tuntas di bagian paling bawah, sebelum akhirnya mencetak nilai node induknya.

baris 69 - 132, syntax int main() berfungsi sebagai titik awal absolut dari seluruh siklus kehidupan program di mana sistem operasi pertama kali menyerahkan kontrol eksekusi prosesor. di dalamnya, program pertama-tama mendeklarasikan variabel pohon dan secara eksplisit menyuntikkannya dengan nilai NULL untuk memastikan ruang memori benar-benar bersih sebelum operasi dimulai. arsitektur utamanya digerakkan oleh struktur perulangan do-while yang secara persisten merender antarmuka menu ke

layar terminal sambil menggunakan perintah `system("clear")` untuk menjaga kebersihan visual. saat pengguna memasukkan opsi, blok percabangan switch-case mengambil alih komando: opsi 1 meminta input angka dan memanggil fungsi tambah, sedangkan opsi 2, 3, dan 4 difungsikan khusus untuk memanggil fungsi traversal yang dilengkapi dengan validasi pengecekan NULL agar tidak terjadi error ketika pohon masih kosong. di setiap akhir putaran eksekusi, layar akan dibekukan sementara waktu menggunakan pemanggilan `getch()` agar pengguna dapat membaca output dengan tenang, hingga akhirnya putaran dihancurkan saat pengguna memilih opsi 5 untuk keluar dan mengembalikan kode `EXIT_FAILURE` secara terhormat.

LINK GITHUB

<https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Modul%207>